

MODULE 1 LAB WORKBOOK

Automation with Selenium

Name – Sristi Saha

Institute – Institute of Engineering & Management, Kolkata

Enrollment No. – 12023052016066

Module	Automation with Selenium
Module Focus	Selenium WebDriver, locators, synchronization, web controls, alerts, tables, windows, tabs, and iframes
Language	Python
Browser	Google Chrome
Automation Framework	Selenium WebDriver
Experiments Covered	Assignment 1 to Assignment 6

Experiment List

Experiment	Title	Primary Concepts
1	The Multi-Locator Challenge	ID, NAME, XPath, URL assertion
2	Synchronization & Explicit Waits	WebDriverWait, expected_conditions
3	Dynamic Dropdowns & Checkboxes	Checkboxes, is_selected(), autocomplete
4	JavaScript Alerts and Confirms	Alert, Confirm, Prompt
5	The HTML Web Table Extractor	Rows, columns, text extraction
6	Windows, Tabs, and Iframes	iframe, window_handles, tab switching

Experiment 1: The Multi-Locator Challenge

Module	Automation with Selenium
Experiment Title	The Multi-Locator Challenge
Practice Website	https://www.saucedemo.com/

Problem Statement

Automate a login page using multiple Selenium locator strategies. The username field must be located by ID, the password field by NAME, and the login button by XPath. The successful login must be validated by checking the resulting URL.

Objective

To practice identifying and interacting with web elements using different locator strategies and to validate a successful browser action using an assertion.

Tools, Software, and Concepts Used

- Python 3
- Selenium WebDriver 4
- Google Chrome
- PyCharm / Python IDE

Key concepts:

- By.ID
- By.NAME
- By.XPATH
- send_keys()
- click()
- WebDriverWait
- EC.url_contains()
- assertion
- try/finally

Implementation Steps

1. Open the SauceDemo login page.
2. Locate the username field using its ID and enter the valid username.
3. Locate the password field using its NAME and enter the password.
4. Locate the login button using XPath and click it.
5. Wait until the URL contains /inventory.html.
6. Assert the expected URL condition and close the browser.

Source Code

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.chrome.service import Service
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
```

```

driver = webdriver.Chrome()

try:
    driver.get("https://www.saucedemo.com/")
    driver.maximize_window()

    username = WebDriverWait(driver, 10).until(
        EC.visibility_of_element_located((By.ID, "user-name")))
    )
    username.send_keys("standard_user")
    password = driver.find_element(By.NAME, "password")
    password.send_keys("secret_sauce")

    login_button = driver.find_element(
        By.XPATH, "//input[@type='submit']"
    )
    login_button.click()

    WebDriverWait(driver, 10).until(
        EC.url_contains("/inventory.html")
    )
    current_url = driver.current_url

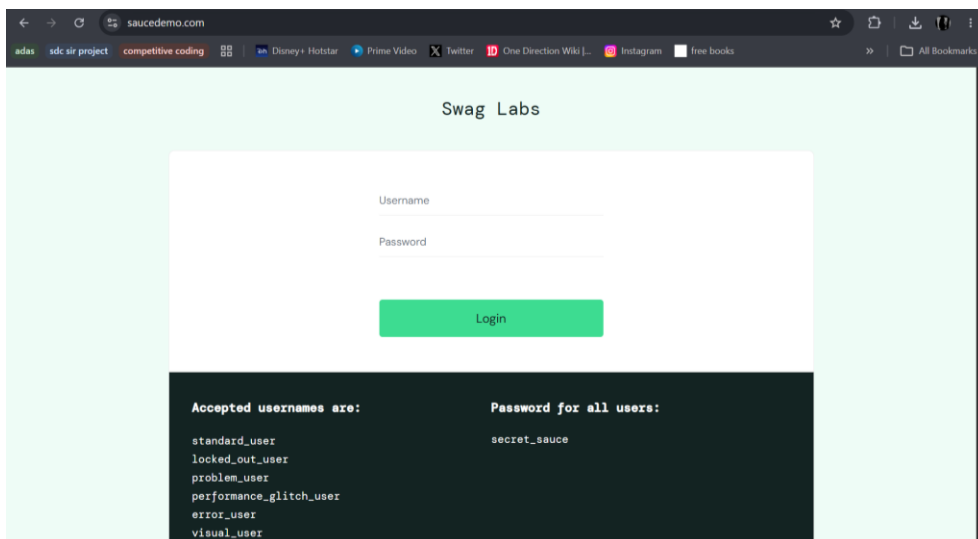
    assert "/inventory.html" in current_url
    print("Assignment 1 PASSED")
    print("Login successful!")
    print("Current URL:", current_url)

finally:
    driver.quit()

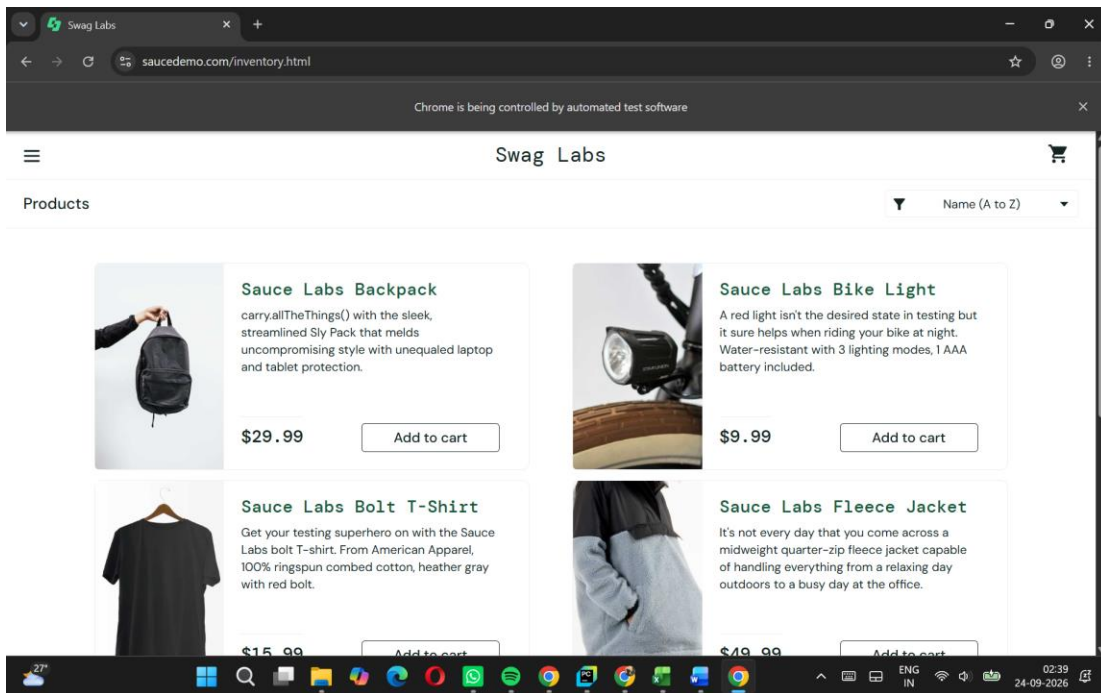
```

Output

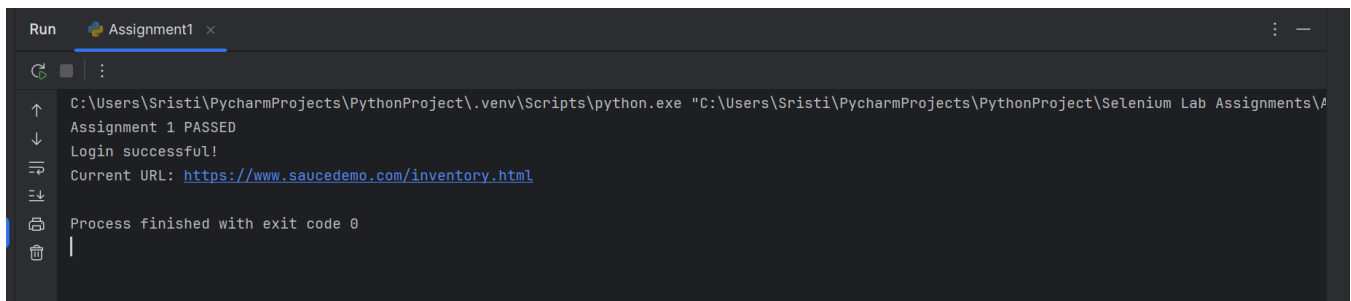
1.1 Site before execution



1.2 Site after execution



1.3 Console output



Result and Observation

The login page was automated successfully using three different locator strategies. The URL assertion confirmed that the browser reached the inventory page after authentication.

Conclusion

The experiment demonstrated practical use of Selenium locators, browser interaction methods, explicit synchronization, and assertions for validation.

Experiment 2: Synchronization & Explicit Waits

Module	Automation with Selenium
Experiment Title	Synchronization & Explicit Waits
Practice Website	https://the-internet.herokuapp.com/dynamic_loading/1

Problem Statement

Automate a page containing dynamically loaded content. The script must not use `time.sleep()`; instead, it must wait explicitly until the required element becomes visible.

Objective

To understand synchronization problems in web automation and implement `WebDriverWait` with `expected_conditions` for reliable interaction with dynamic elements.

Tools, Software, and Concepts Used

- Python 3
- Selenium WebDriver 4
- Google Chrome
- PyCharm / Python IDE

Key concepts:

- `WebDriverWait`
- `expected_conditions`
- `element_to_be_clickable`
- `visibility_of_element_located`
- dynamic content
- explicit synchronization

Implementation Steps

1. Open the dynamic-loading page.
2. Wait until the Start button is clickable.
3. Click the Start button.
4. Wait until the dynamically generated text element becomes visible.
5. Extract the text and verify that it is Hello World!.
6. Print the successful result and close the browser.

Source Code

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
driver = webdriver.Chrome()

try:
    driver.get(
        "https://the-internet.herokuapp.com/dynamic_loading/1"
    )
    driver.maximize_window()
```

```
print("Page opened successfully.")

start_button = WebDriverWait(driver, 10).until(
    EC.element_to_be_clickable(
        (By.XPATH, "//button[text()='Start']")
    )
)
print("Start button found.")
start_button.click()
print("Waiting for dynamic content...")

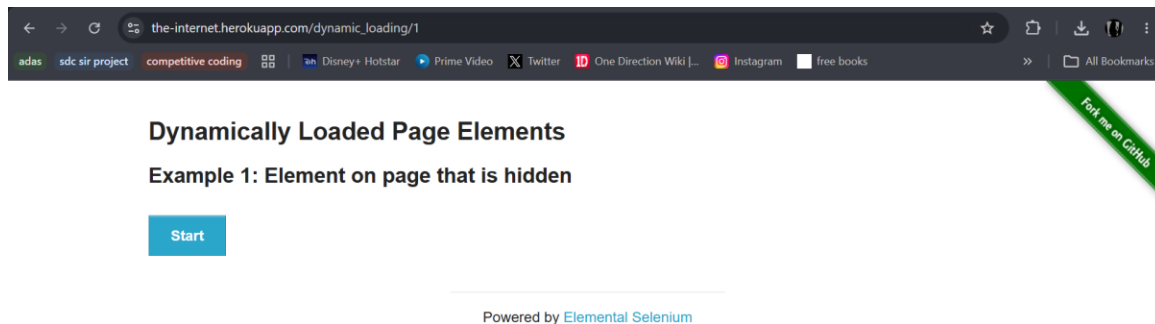
text_element = WebDriverWait(driver, 15).until(
    EC.visibility_of_element_located(
        (By.XPATH, "//div[@id='finish']/h4")
    )
)

text = text_element.text
print("Dynamic content:", text)
assert text == "Hello World!"
print("Assignment 2 PASSED")

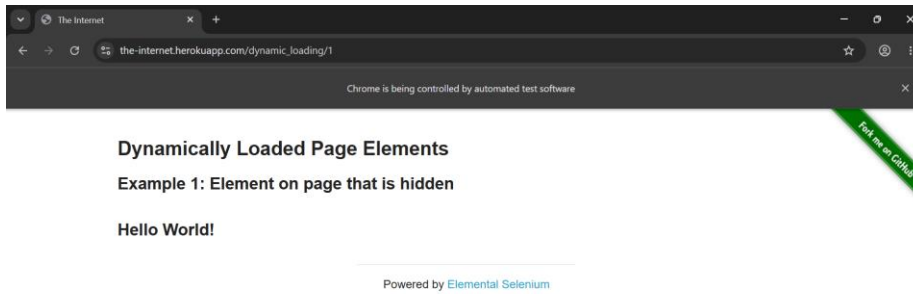
finally:
    driver.quit()
```

Output

2.1 Dynamic loading page before clicking Start



2.2 Hello World! Displayed after loading



2.3 Console output



Result and Observation

The dynamic text was retrieved without using `time.sleep()`. `WebDriverWait` waited only until the required condition was satisfied, demonstrating reliable explicit synchronization.

Conclusion

The experiment demonstrated why explicit waits are preferred over fixed delays when working with dynamically changing web pages.

Experiment 3: Dynamic Dropdowns & Checkboxes

Module	Automation with Selenium
Experiment Title	Dynamic Dropdowns & Checkboxes
Practice Website	https://demoqa.com/automation-practice-form

Problem Statement

Automate a registration form containing multiple checkboxes and a search-as-you-type autocomplete field. The selected state of the checkboxes must be verified and a matching autocomplete option must be selected.

Objective

To practice checkbox handling, state verification using `is_selected()`, dynamic suggestion handling, CSS selectors, and iteration through `WebElements`.

Tools, Software, and Concepts Used

- Python 3
- Selenium WebDriver 4
- Google Chrome
- PyCharm / Python IDE

Key concepts:

- Checkbox selection
- `is_selected()`
- XPath
- CSS selectors
- autocomplete
- `find_elements()`
- `WebDriverWait`
- looping through `WebElements`
- dynamic DOM handling

Implementation Steps

1. Open the DemoQA automation practice form.
2. Locate and select the Sports checkbox.
3. Verify that the Sports checkbox is selected.
4. Locate and select the Reading checkbox.
5. Verify that the Reading checkbox is selected.
6. Type Math into the Subjects autocomplete field.
7. Wait for suggestions and iterate through them.
8. Select the Maths suggestion and print the successful result.

Source Code

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
```

```

driver = webdriver.Chrome()
wait = WebDriverWait(driver, 10)

try:
    driver.get("https://demoqa.com/automation-practice-form")
    driver.maximize_window()
    print("Registration page opened.")

    #select checkboxes
    sports_checkbox = wait.until(
        EC.element_to_be_clickable(
            (By.XPATH, "//label[text()='Sports']")
        )
    )

    sports_checkbox.click()
    sports_input = driver.find_element(
        By.ID, "hobbies-checkbox-1"
    )
    print("Sports selected:", sports_input.is_selected())

    reading_checkbox = wait.until(
        EC.element_to_be_clickable(
            (By.XPATH, "//label[text()='Reading']")
        )
    )
    reading_checkbox.click()
    reading_input = driver.find_element(
        By.ID, "hobbies-checkbox-2"
    )
    print("Reading selected:", reading_input.is_selected())

    assert sports_input.is_selected(), \
        "Sports checkbox was not selected."

    assert reading_input.is_selected(), \
        "Reading checkbox was not selected."

    # autocomplete dropdown
    subject_input = wait.until(
        EC.element_to_be_clickable(
            (By.ID, "subjectsInput")
        )
    )

    subject_input.send_keys("Math")
    print("Typed 'Math' into autocomplete field.")
    wait.until(
        EC.visibility_of_element_located(
            (
                By.CSS_SELECTOR,
                ".subjects-auto-complete__option"
            )
        )
    )

```

```

    )
    )
)

suggestions = driver.find_elements(
    By.CSS_SELECTOR,
    ".subjects-auto-complete__option"
)
print("Suggestions found:")
target_option = "Maths"
for suggestion in suggestions:
    suggestion_text = suggestion.text.strip()
    print("-", suggestion_text)
    if suggestion_text == target_option:
        suggestion.click()
        print(
            "Matching option selected:",
            suggestion_text
        )
        break
    else:
        raise Exception(
            f"Autocomplete option '{target_option}' "
            f"was not found."
        )
print("Assignment 3 PASSED")

finally:
    driver.quit()

```

Output / Screenshots

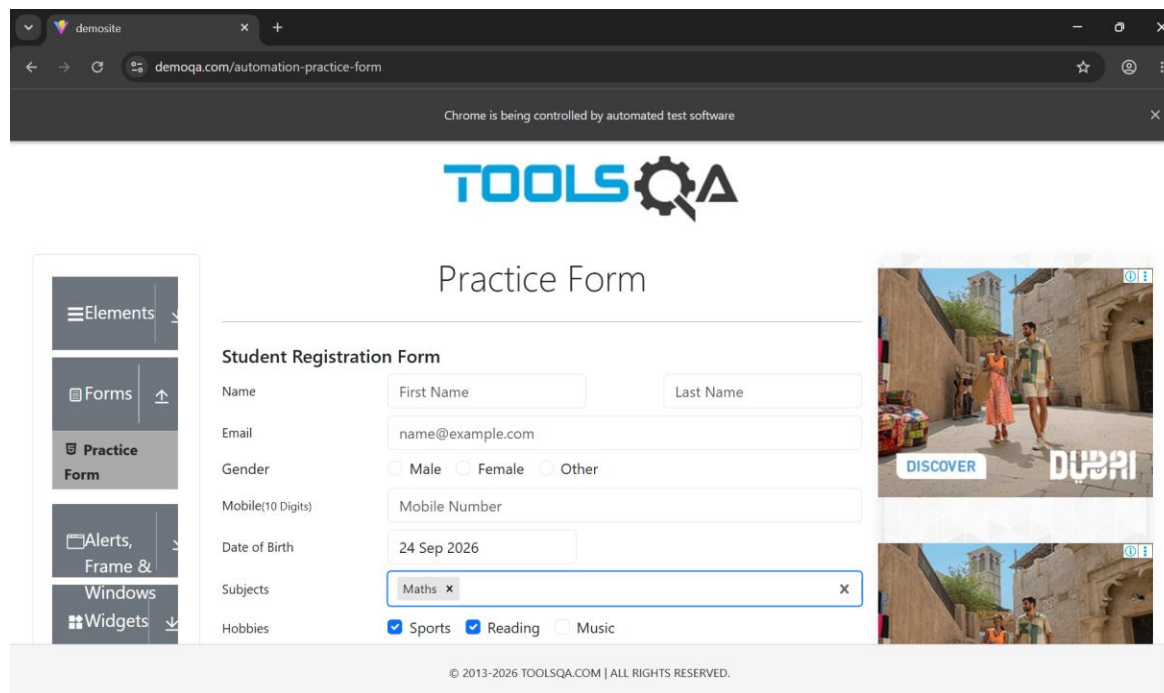
3.1 DemoQA form before script execution

The screenshot shows a web browser window with the URL `demoqa.com/automation-practice-form`. The page features the **TOOLSQA** logo at the top center. On the left, there is a sidebar with navigation links: **Elements**, **Forms**, **Practice Form** (highlighted), **Alerts, Frame & Windows**, and **Widgets**. The main content area is titled **Practice Form** and contains a **Student Registration Form**. The form fields are as follows:

- Name:** Two input fields for **First Name** and **Last Name**.
- Email:** One input field containing the text `name@example.com`.
- Gender:** Three radio buttons labeled **Male**, **Female**, and **Other**.
- Mobile(10 Digits):** One input field for **Mobile Number**.
- Date of Birth:** One input field containing the text `24 Sep 2026`.
- Subjects:** One input field.
- Hobbies:** Three checkboxes labeled **Sports**, **Reading**, and **Music**.
- Picture:** A file selection area with a **Choose File** button and the text **No file chosen**.

On the right side of the form, there are two promotional banners. The top banner says **Free for students** with the text "Gemini Notebook is making the grade" and a right arrow icon. The bottom banner says **Start presentations in a snap** with the text "Click the blank page - Transform". At the bottom of the page, there is a footer that reads: **© 2013-2026 TOOLSQA.COM | ALL RIGHTS RESERVED.**

3.2 DemoQA form with Sports and Reading selected and Maths autocomplete suggestion



The screenshot shows a web browser window with the URL `demoqa.com/automation-practice-form`. The page title is "Practice Form" and it features the "TOOLSQA" logo. On the left, there is a sidebar with navigation links: "Elements", "Forms", "Practice Form", "Alerts, Frame & Windows", and "Widgets". The main content area is titled "Student Registration Form" and contains the following fields and options:

- Name:** First Name and Last Name input fields.
- Email:** Input field with the value `name@example.com`.
- Gender:** Radio buttons for Male, Female, and Other.
- Mobile(10 Digits):** Input field with the value `Mobile Number`.
- Date of Birth:** Input field with the value `24 Sep 2026`.
- Subjects:** A dropdown menu showing `Maths` as a suggestion.
- Hobbies:** Checkboxes for Sports, Reading, and Music. Both Sports and Reading are selected.

At the bottom of the form, there is a copyright notice: "© 2013-2026 TOOLSQA.COM | ALL RIGHTS RESERVED."

3.3 Console Output



```
Run Assignment3 x
C:\Users\Sristi\PycharmProjects\PythonProject\.venv\Scripts\python.exe "C:\Users\Sristi\PycharmProjects\PythonProject\Selenium Lab Assignments\Assignment3.py"
Registration page opened.
Sports selected: True
Reading selected: True
Typed 'Math' into autocomplete field.
Suggestions found:
- Maths
Matching option selected: Maths
Assignment 3 PASSED

Process finished with exit code 0
```

Result and Observation

Both checkboxes were selected and verified using `is_selected()`. The autocomplete field produced a dynamic suggestion, and the script located and selected the required Maths option. The suggestion text was stored before clicking to avoid a stale element reference after the DOM update.

Conclusion

The experiment demonstrated practical automation of checkboxes and dynamic autocomplete controls, including synchronization and handling of changing DOM elements.

Experiment 4: JavaScript Alerts and Confirms

Module	Automation with Selenium
Experiment Title	JavaScript Alerts and Confirms
Practice Website	https://the-internet.herokuapp.com/javascript_alerts

Problem Statement

Automate three JavaScript dialog types: an alert, a confirm box, and a prompt box. The alert must be accepted, the confirm dismissed, and text entered into the prompt before it is submitted.

Objective

To learn how Selenium switches from the normal page context to JavaScript alert dialogs and performs accept, dismiss, and text-entry operations.

Tools, Software, and Concepts Used

- Python 3
- Selenium WebDriver 4
- Google Chrome
- PyCharm / Python IDE

Key concepts:

- `driver.switch_to.alert`
- `alert.text`
- `accept()`
- `dismiss()`
- `send_keys()`
- `alert_is_present()`
- JavaScript dialogs

Implementation Steps

1. Open the JavaScript Alerts page.
2. Click the JS Alert button and wait for the alert.
3. Read its message and accept it.
4. Click the JS Confirm button and dismiss the confirm dialog.
5. Click the JS Prompt button and wait for the prompt.
6. Enter Selenium Test using `send_keys()`.

Source Code

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

driver = webdriver.Chrome()
```

```
wait = WebDriverWait(driver, 10)
```

```
try:
```

```
    driver.get("https://the-internet.herokuapp.com/javascript_alerts")
    driver.maximize_window()
    print("JavaScript Alerts page opened.")
```

```
    # Javascript alert
```

```
    alert_button = wait.until(
        EC.element_to_be_clickable(
            (By.XPATH, "//button[text()='Click for JS Alert']")
        )
    )
```

```
    alert_button.click()
    alert = wait.until(EC.alert_is_present())
    print("Alert message:", alert.text)
    alert.accept()
    print("Alert accepted.")
```

```
    #javascript confirm
```

```
    confirm_button = wait.until(
        EC.element_to_be_clickable(
            (By.XPATH, "//button[text()='Click for JS Confirm']")
        )
    )
```

```
    confirm_button.click()
    confirm = wait.until(EC.alert_is_present())
    print("Confirm message:", confirm.text)
    confirm.dismiss()
    print("Confirm dismissed.")
```

```
    # Javascript prompt
```

```
    prompt_button = wait.until(
        EC.element_to_be_clickable(
            (By.XPATH, "//button[text()='Click for JS Prompt']")
        )
    )
```

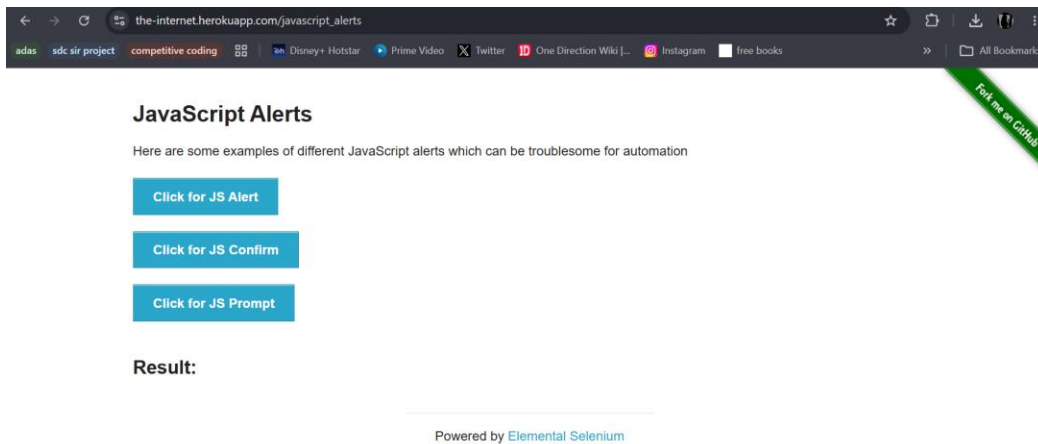
```
    prompt_button.click()
    prompt = wait.until(EC.alert_is_present())
    print("Prompt message:", prompt.text)
    prompt.send_keys("Selenium Test")
    print("Text entered into prompt.")
    prompt.accept()
    print("Prompt accepted.")
    print("Assignment 4 PASSED")
```

```
finally:
```

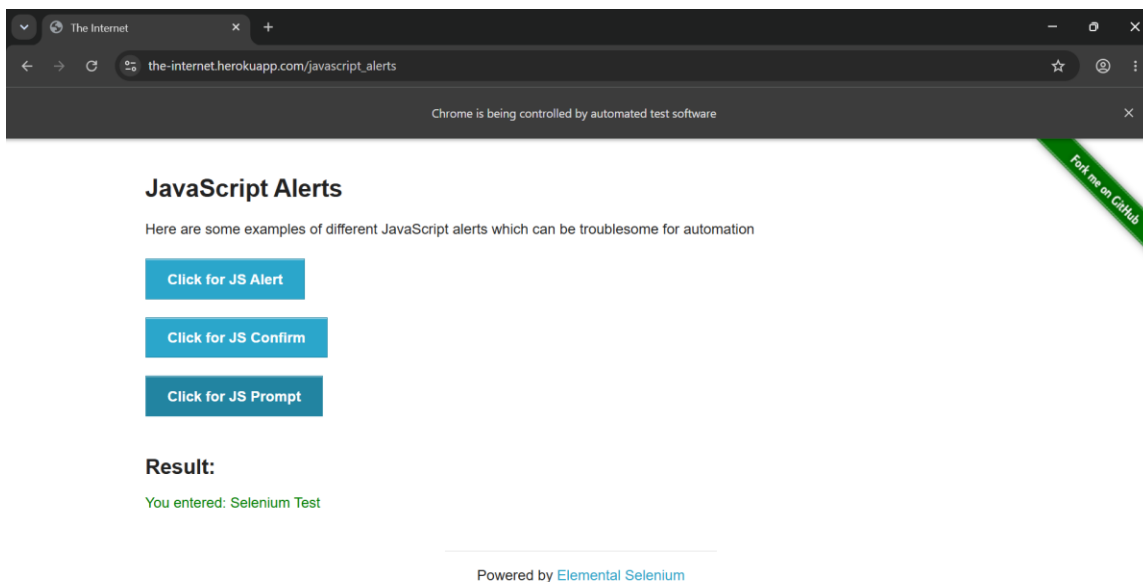
```
    driver.quit()
```

Output / Screenshots

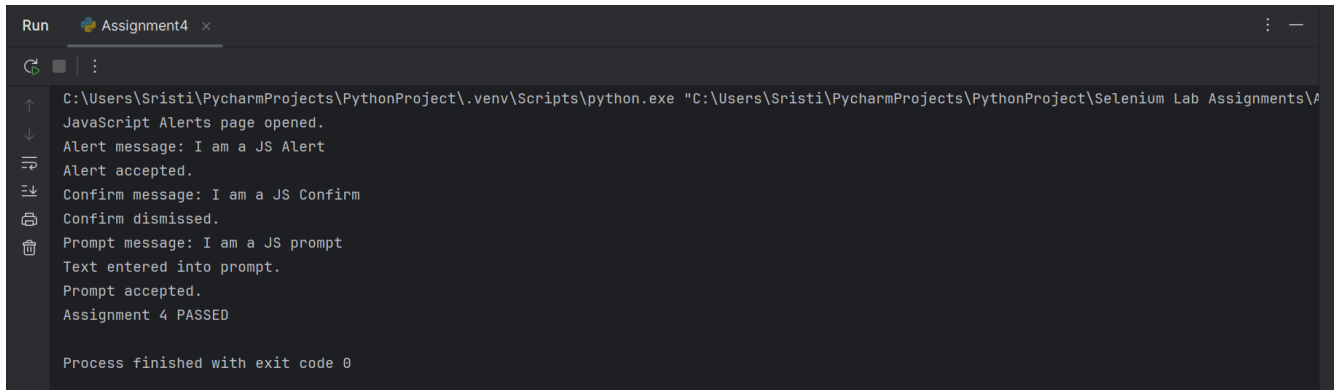
4.1 Site before execution of script



4.2 JavaScript Alert and Confirm displayed and Prompt entered



4.3 Console Output



```
Run Assignment4 x
C:\Users\Sristi\PycharmProjects\PythonProject\.venv\Scripts\python.exe "C:\Users\Sristi\PycharmProjects\PythonProject\Selenium Lab Assignments\JavaScript Alerts page opened.
Alert message: I am a JS Alert
Alert accepted.
Confirm message: I am a JS Confirm
Confirm dismissed.
Prompt message: I am a JS prompt
Text entered into prompt.
Prompt accepted.
Assignment 4 PASSED

Process finished with exit code 0
```

Result and Observation

All three JavaScript dialog types were successfully handled. The alert was accepted, the confirm was dismissed, and text was entered into the prompt before accepting it.

Conclusion

The experiment demonstrated Selenium's alert-handling API and the difference between accepting, dismissing, and entering data into JavaScript dialogs.

Experiment 5: The HTML Web Table Extractor

Module	Automation with Selenium
Experiment Title	The HTML Web Table Extractor
Practice Website	https://the-internet.herokuapp.com/tables

Problem Statement

Automate a multi-column HTML data table. The script must iterate through rows and columns, display the table data, identify a particular row by matching a name, and retrieve the corresponding Due value from that row.

Objective

To understand the structure of HTML tables and use Selenium to traverse headers, rows, and individual table cells.

Tools, Software, and Concepts Used

- Python 3
- Selenium WebDriver 4
- Google Chrome
- PyCharm / Python IDE

Key concepts:

- Web tables
- find_elements()
- CSS selectors
- Tag Name locator
- rows and columns
- .text
- nested element search
- row matching

Implementation Steps

7. Open the sortable tables page.
8. Locate table1.
9. Read and print all table headers.
10. Locate all body rows.
11. Iterate through every row and its cells and print the data.
12. Search for the row whose Last Name is Smith.
13. Retrieve the Due value from the matching row.
14. Display the extracted value and finish the experiment.

Source Code

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
```

```

driver = webdriver.Chrome()
wait = WebDriverWait(driver, 10)

try:
    driver.get("https://the-internet.herokuapp.com/tables")
    driver.maximize_window()
    print("Web table page opened.")

    #locate table
    table = wait.until(
        EC.visibility_of_element_located(
            (By.ID, "table1")
        )
    )

    # read table headers
    headers = table.find_elements(
        By.CSS_SELECTOR,
        "thead th"
    )

    header_names = []

    for header in headers:
        header_names.append(header.text.strip())

    print("\nTable Headers:")
    print(header_names)

    # read all rows
    rows = table.find_elements(
        By.CSS_SELECTOR,
        "tbody tr"
    )
    print("\nTable Data:")

    for row in rows:
        columns = row.find_elements(
            By.TAG_NAME,
            "td"
        )

        row_data = []
        for column in columns:
            row_data.append(column.text.strip())
        print(row_data)

    # find specific person
    target_name = "Smith"
    target_due = None
    for row in rows:
        columns = row.find_elements(

```

```

        By.TAG_NAME,
        "td"
    )

    last_name = columns[0].text.strip()

    if last_name == target_name:
        target_due = columns[3].text.strip()
        break

    if target_due is not None:
        print("\nTarget person found:", target_name)
        print("Due value:", target_due)

    else:
        print("\nPerson not found:", target_name)
        print("Assignment 5 PASSED")

finally:
    driver.quit()

```

Output / Screenshots

5.1 Web table displayed in browser

Data Tables

Often times when you see a table it contains data which is sortable -- sometimes with actions that can be taken within each row (e.g. edit, delete). And it can be challenging to automate interaction with sets of data in a table depending on how it is constructed.

Example 1

No Class or ID attributes to signify groupings of rows and columns

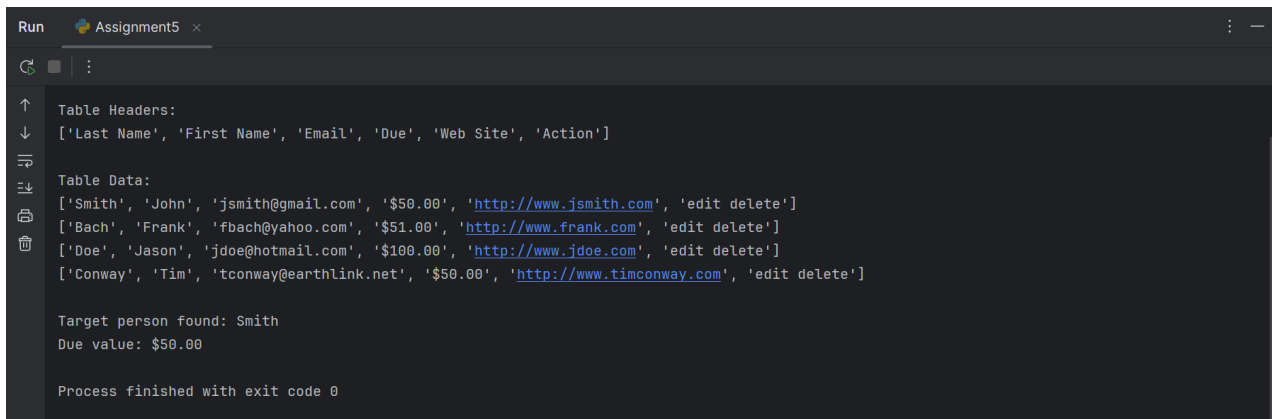
Last Name	First Name	Email	Due	Web Site	Action
Smith	John	jsmith@gmail.com	\$50.00	http://www.jsmith.com	edit delete
Bach	Frank	fbach@yahoo.com	\$51.00	http://www.frank.com	edit delete
Doe	Jason	jdoe@hotmail.com	\$100.00	http://www.jdoe.com	edit delete
Conway	Tim	tconway@earthlink.net	\$50.00	http://www.timconway.com	edit delete

Example 2

Class and ID attributes to signify groupings of rows and columns

Last Name	First Name	Email	Due	Web Site	Action
Smith	John	jsmith@gmail.com	\$50.00	http://www.jsmith.com	edit delete
Bach	Frank	fbach@yahoo.com	\$51.00	http://www.frank.com	edit delete
Doe	Jason	jdoe@hotmail.com	\$100.00	http://www.jdoe.com	edit delete

5.2 Console Output



```
Run Assignment5 x
Table Headers:
['Last Name', 'First Name', 'Email', 'Due', 'Web Site', 'Action']

Table Data:
['Smith', 'John', 'jsmith@gmail.com', '$50.00', 'http://www.jsmith.com', 'edit delete']
['Bach', 'Frank', 'fbach@yahoo.com', '$51.00', 'http://www.frank.com', 'edit delete']
['Doe', 'Jason', 'jdoe@hotmail.com', '$100.00', 'http://www.jdoe.com', 'edit delete']
['Conway', 'Tim', 'tconway@earthlink.net', '$50.00', 'http://www.timconway.com', 'edit delete']

Target person found: Smith
Due value: $50.00

Process finished with exit code 0
```

Result and Observation

The script successfully located the HTML table, extracted its headers, traversed each row and column, and retrieved the Due value associated with the row whose Last Name matched Smith.

Conclusion

The experiment demonstrated how Selenium can be used to traverse structured HTML data and extract a value from the same row as a matching record.

Experiment 6: Windows, Tabs, and Iframes

Module	Automation with Selenium
Experiment Title	Windows, Tabs, and Iframes
Practice Website	https://the-internet.herokuapp.com/iframe and https://the-internet.herokuapp.com/windows

Problem Statement

Automate a page containing an embedded iframe and a separate page containing a link that opens a new browser window/tab. The script must switch into the iframe, interact with its editor, switch back, open the new window, obtain its title, close it, and return to the original window.

Objective

To learn how Selenium manages different browsing contexts, including iframe contexts and multiple browser windows/tabs.

Tools, Software, and Concepts Used

- Python 3
- Selenium WebDriver 4
- Google Chrome
- PyCharm / Python IDE

Key concepts:

- `switch_to.frame()`
- `switch_to.default_content()`
- `current_window_handle`
- `window_handles`
- `switch_to.window()`
- `close()`
- `iframe`
- multiple windows/tabs
- `Keys.CONTROL`

Implementation Steps

15. Open the iframe page.
16. Locate the iframe and switch Selenium into it.
17. Locate the TinyMCE editor.
18. Click the editor, select its existing content using Ctrl+A, delete it, and enter new text.
19. Switch back to the main document.
20. Open the Windows page and store the original window handle.
21. Click the link that opens the new window.
22. Wait for two window handles and switch to the new window.
23. Read and print its title.
24. Close the new window and switch back to the original window.

Source Code

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.common.keys import Keys
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
```

```
driver = webdriver.Chrome()
wait = WebDriverWait(driver, 10)
```

```
try:
    #iframe
    driver.get(
        "https://the-internet.herokuapp.com/iframe"
    )
    driver.maximize_window()
    print("Iframe page opened.")
```

```
    iframe = wait.until(
        EC.presence_of_element_located(
            (By.ID, "mce_0_ifr")
        )
    )
```

```
    driver.switch_to.frame(iframe)
    print("Switched into iframe.")
    editor = wait.until(
        EC.element_to_be_clickable(
            (By.ID, "tinymce")
        )
    )
```

```
    editor.click()
    editor.send_keys(
        Keys.CONTROL,
        "a"
    )
    editor.send_keys(
        Keys.BACKSPACE
    )
```

```
    editor.send_keys(
        "Hello from Selenium!"
    )
```

```
    print("Text entered inside iframe.")
```

```
    driver.switch_to.default_content()
```

```
    print("Switched back to main page.")
```

```
# new tab/window
```

```
driver.get(
    "https://the-internet.herokuapp.com/windows"
)
print("\nWindows page opened.")

original_window = driver.current_window_handle

new_window_button = wait.until(
    EC.element_to_be_clickable(
        (By.LINK_TEXT, "Click Here")
    )
)

new_window_button.click()
wait.until(
    EC.number_of_windows_to_be(2)
)

windows = driver.window_handles
print(
    "Number of windows:",
    len(windows)
)

for window in windows:
    if window != original_window:
        driver.switch_to.window(window)
        break

new_window_title = driver.title

print("New window title:", new_window_title)

driver.close()
print("New window closed.")

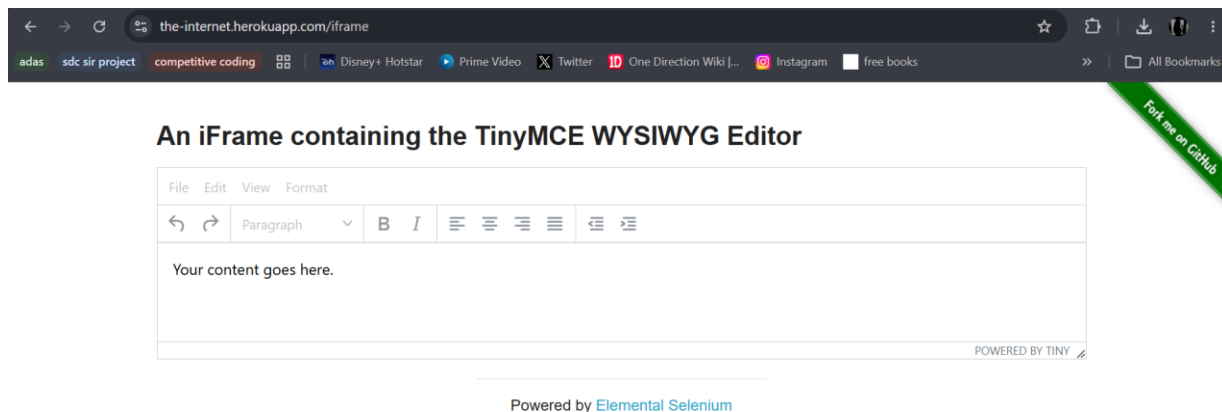
driver.switch_to.window(
    original_window
)

print("Switched back to original window.")
print("\nAssignment 6 PASSED")

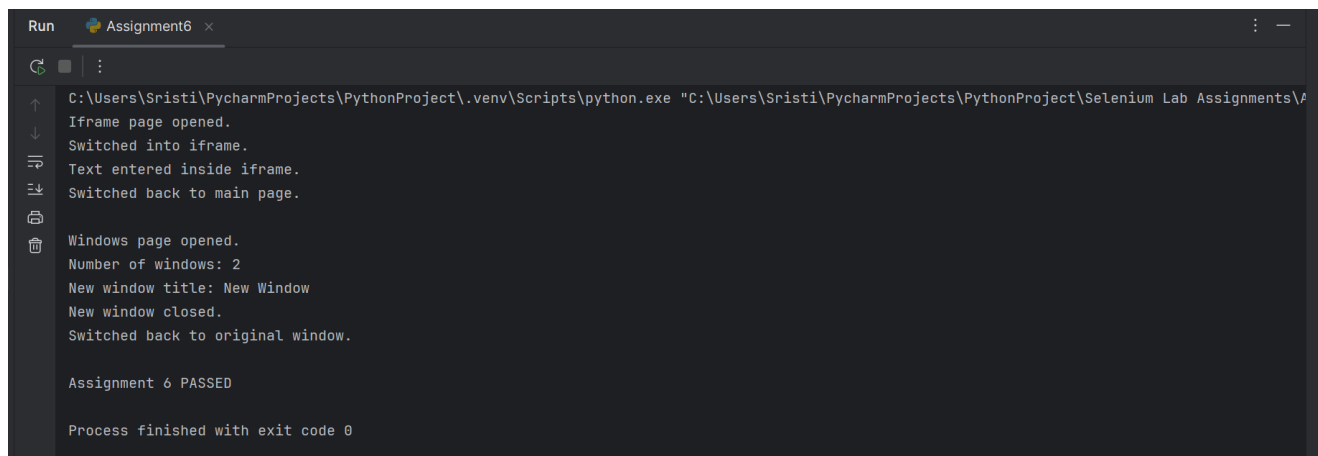
finally:
    driver.quit()
```

Output

6.1 Iframe page before execution



6.2 Console output after execution



Result and Observation

The iframe was accessed by switching Selenium into the iframe context, and the TinyMCE editor was updated successfully. The script then handled a second browser context, retrieved the new window title, closed the new window, and returned to the original window.

Conclusion

The experiment demonstrated context switching in Selenium for both iframes and multiple browser windows/tabs. It also demonstrated keyboard actions for a contenteditable editor.

Overall Module Result

The Automation with Selenium module was completed through six practical experiments covering the core and advanced WebDriver concepts specified in the lab assignment. The experiments progressed from basic element identification and synchronization to dynamic controls, JavaScript dialogs, web-table traversal, iframe interaction, and multiple browser-window handling.

Overall Observation

- Different locator strategies are required depending on the structure and attributes of web elements.
- Explicit waits improve synchronization with dynamic web pages and avoid unnecessary fixed delays.
- Checkbox state can be verified programmatically using `is_selected()`.
- Dynamic autocomplete components may re-render their DOM, so stale element references must be handled carefully.
- JavaScript alerts, confirms, and prompts require switching Selenium's context to the alert object.
- HTML tables can be traversed systematically by first locating the table, then its rows and cells.
- Iframes require switching into the frame before interacting with their internal elements.
- Multiple tabs or windows require window handles and explicit context switching.

Overall Conclusion

The experiments provided hands-on understanding of Selenium WebDriver automation using Python. The completed work demonstrates the ability to identify web elements, synchronize with dynamic content, operate complex controls, extract structured data, and manage multiple browser contexts. These concepts form the practical foundation for developing reliable browser automation scripts.